Project 3: Generative art

Overview:The project aims to help students understand object-oriented programming, work with external libraries, and explore creative applications of coding. It's focused on creating generative art using Pythong and the PIL library.

Materials:

<aside> 👉 This project won't have starter code.

</aside>

- Replit.
- Student Brief
- Simple answer:https://replit.com/@VerPasamic1/Project-3-test#main.py

---

Before the lesson:

-

During the lesson:

Step 1: Set Up Your Project Environment

- **Objective:** Prepare your coding environment and install necessary packages.
- **Actions:**
  - Ensure Python and pip are installed on all student machines.
  - Guide students to install the PIL library (Pillow) using the command `pip install Pillow`.

Step 2: Defining the ArtElement Class

- **Objective:** Teach students to create a base class for art elements using dictionaries for dynamic attribute storage.
- **Actions:**
  - Introduce the concept of using dictionaries to manage flexible attributes of art elements.
  - Guide students to implement an `ArtElement` class that can update and utilize these attributes for drawing.
  - Example Code

```
from PIL import Image, ImageDraw

class ArtElement:
    def __init__(self, attributes):
        self.attributes = attributes

    def update_attribute(self, key, value):
        self.attributes[key] = value
```

```
    def draw(self, draw_context):
        if 'position' in self.attributes and 'size' in self.attributes and 'color' in self.att
            position = self.attributes['position']
            size = self.attributes['size']
            color = self.attributes['color']
            upper_left = (position[0] - size, position[1] - size)
            lower_right = (position[0] + size, position[1] + size)
            draw_context.ellipse([upper_left, lower_right], fill=color)
```

Step 3: Creating the Canvas Class

- **Objective:** Develop a container class for managing multiple art elements.
- **Actions:**
    - Explain the structure of a digital canvas and its role in organizing art elements.
    - Assist students in creating a `Canvas` class that manages a collection of `ArtElement` objects and facilitates rendering.
    - Example Code:

```
class Canvas:
    def __init__(self, size, background_color='white'):
        self.image = Image.new('RGB', size, background_color)
        self.draw_context = ImageDraw.Draw(self.image)
        self.elements = []

    def add_element(self, element):
        self.elements.append(element)

    def render(self):
        for element in self.elements:
            element.draw(self.draw_context)
        return self.image
```

Step 4: Composing and Generating Artwork

- **Objective:** Apply the classes to generate unique generative art.
- **Actions:**
    - Guide students through writing a function to create and arrange multiple `ArtElement` instances with random attributes.
    - Demonstrate how to use loops and randomization to generate diverse artworks.
    - Example Code:

```
import random

def generate_artwork():
    canvas = Canvas((800, 600), 'black')

    for _ in range(50):
        position = (random.randint(0, 800), random.randint(0, 600))
        size = random.randint(10, 50)
        color = (random.randint(0, 255), random.randint(0, 255), random.randint(0, 255))
        circle_attributes = {'position': position, 'size': size, 'color': color}
        circle = ArtElement(circle_attributes)
```

```
        canvas.add_element(circle)

    final_art = canvas.render()
    final_art.show()
    final_art.save('my_generative_artwork.png')

if __name__ == "__main__":
    generate_artwork()
```

Step 5: Rendering and Displaying the Final Artwork

- **Objective:** Show students how to finalize and present their generative art.
- **Actions:**
    - Instruct students on rendering their canvas to produce the final artwork.
    - Show how to use Pillow's functions to display and save the generated image.

By following these steps, students will not only learn about Python programming and working with graphical libraries but also explore creative expression through code. This project offers an engaging way to combine technology and art, inspiring students to apply programming skills in innovative ways.

Extensions:

-
